Exno: 5
Date : 23/09/2026

Programs on Decision Making and Knowledge Representation

AIM:
To write and execute simple Prolog programs for decision making and knowledge representation.

A) Introduction To Prolog

Prolog (Programing In Logic)

• PROLOG is declarative- specify facts and logical relationships
• Non-procedural
• Data base language with automated search and the ability to follow general rules.
• Symbolic - symbols are used to represent object
• High level - contains a built-in problem solving mechanism
1. PROLOG Programming (When we program in Prolog we need to provide following three things declaring some facts about objects and their relationships.
2. Defining some rules about objects and their relationships.
3. Asking questions about objects and their relationships.

Prolog Syntax

Terms:
The central data structure in PROLOG is that of a term. There are terms of four kinds:
• atoms,
• numbers,
• variables, and
• Compound terms.

Atoms and numbers are sometimes grouped together and called atomic terms.

Atomic Terms:
Atomic terms are usually strings made up of lower- and uppercase letters, digits, and the underscore, starting with a lowercase letter.
The following are all valid PROLOG atoms:
• elephant,
• b,
• abcXYZ,
• x_123,

Compound Terms:
• Compound terms are made up of a PROLOG atom and a number of arguments (PROLOG terms, i.e., atoms, numbers, variables, or other compound terms) enclosed in parentheses and separated by commas.

• The following are some examples for compound terms: o
is_bigger(horse, X), f(g(X, _), 7),
• The sets of compound terms and atoms together form the set of PROLOG predicates.
• A term that doesn't contain any variables is called a ground term.
Variables:
Variables are strings of letters, digits, and the underscore, starting with a capital letter or an underscore.
Examples:
• X,
• Elephant,
• _4711,
• X_1_2,
• MyVariable,
Facts:
A fact is a predicate followed by a dot.
Examples:
• bigger_animal(whale).
• life_is_beautiful.
• The intuitive meaning of a fact is that we define a certain instance of a relation as being true.
Rules:
• A rule consists of a head (a predicate) and a body (a sequence of predicates separated by commas).
• Head and body are separated by the sign :- and, like every PROLOG expression, a rule has to be terminated by a dot.
Examples:
is_smaller(X, Y) :- is_bigger(Y, X).
aunt(Aunt, Child) :- sister(Aunt, Parent),parent(Parent, Child).

• The intuitive meaning of a rule is that the goal expressed by its head is true, if we (or rather the PROLOG system) can show that all of the expressions (subgoals) in the rule's body are true.
Syntax of rule:
<head> :- <body>
Read :- as if.
->likes(john,X) :- likes(X,cricket).

~ "John likes X if X likes cricket".
i.e., "John likes anyone who likes cricket".
• Rules always end with'.'
Programs and Queries:
Programs-
A PROLOG program is a sequence of clauses.

Queries –
• After compilation a PROLOG program is run by submitting queries to the interpreter.
• A query has the same structure as the body of a rule, i.e. it is a sequence of predicates separated by commas and terminated by a dot.
• They can be entered at the PROLOG prompt, which in most implementations looks something like this: ?-.
• When writing about queries we often include the ?.
• Examples:
?- is_bigger(elephant, donkey).
?- small(X), green(X), slimy(X).
• Intuitively, when submitting a query like the last example, we ask PROLOG whether all its predicates are probably true, or in other words whether there is an X such that small(X), green(X), and slimy(X) are all true.
There are two types of queries:
1) Ground Query
• edge(a,h)
• This query is called a ground query because it consists only of value identifier as parameter to the predicate.
• A ground query is posed we expect a yes/no answer.

2) Non-Ground Query
They have variables as parameters
• edge(a,X)
Variables:
• Always begin with a capital letter
• ?- likes (john,X).
• ?- likes (john, Something).
• But not
• ?- likes (john,something)

Install SWI-Prolog:
Steps to Compile:
1. Open Notepad.
2. Write your Facts and Rules and save the file with .pl extension.
3. To open the desired file
File -> Edit -> Go to the location-> select file ->Open
4. To check the syntax
In file editor
Compile ->make If
no error it will
compile
5. To fire query
In PROLOG editor

File -> consult -> select the file name
Write the query, do not forget to put full stop (.) If there is more than one answer to get use (;)

RESULT

Thus, the Prolog programs were successfully executed and the expected results were obtained.